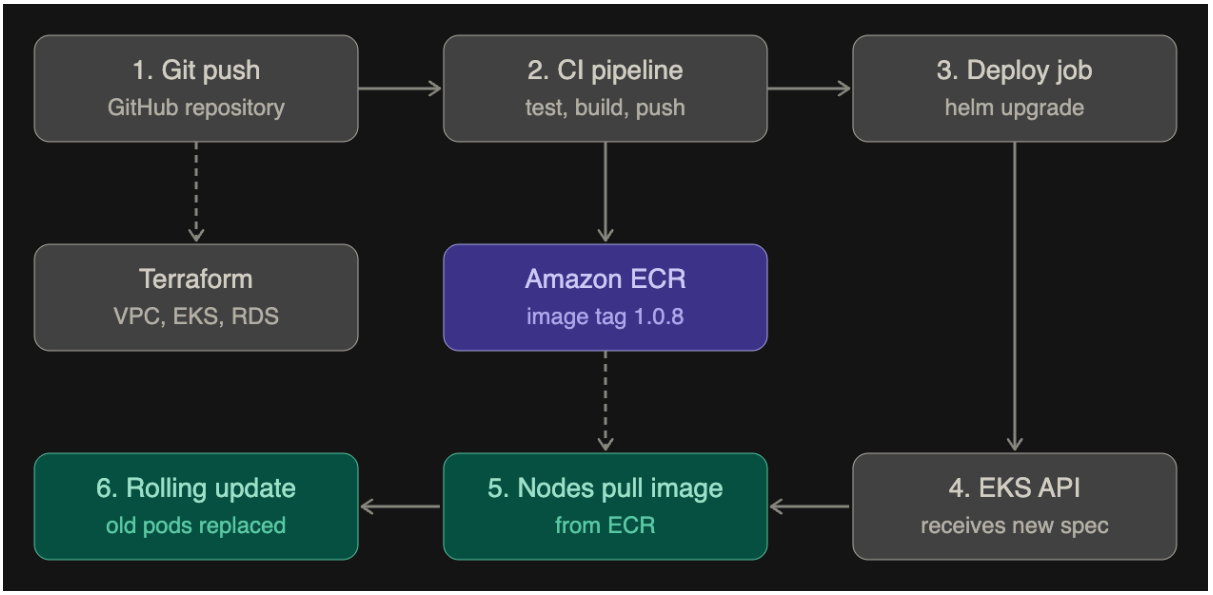
Here are two diagrams for a production version on AWS: the runtime architecture first, then the release pipeline. I used EKS, since your project already runs on Kubernetes, with MySQL moved to Amazon RDS.





The dashed arrows are one-off or pull actions: Terraform builds the infrastructure only when it changes, and the nodes pull from ECR only when new pods start.

How your local setup maps to AWS:

Docker Desktop (now)	AWS (production)
Hosts file, `app.local`	Route 53 plus an ACM certificate
ingress-nginx	ALB, created by the AWS Load Balancer Controller
Docker Desktop node	EKS managed node group (EC2, Auto Scaling)
MySQL pod plus PVC	Amazon RDS for MySQL, Multi-AZ
`kubectl create secret`	Secrets Manager, read by pods through IRSA
Docker Hub	Amazon ECR
Manual `kubectl apply`	CI/CD pipeline (GitHub Actions or CodePipeline)

Where each AWS services:

- **EC2:** the worker nodes that run your pods when you use EKS. You don't manage them one by one. A managed node group handles patching and scaling.
- **EBS:** disk storage for EC2. In this design it is the node root volume, and RDS uses it behind the scenes. You would only attach EBS to pods (through the EBS CSI driver) if you kept MySQL inside Kubernetes.
- **RDS:** the managed database. It replaces your MySQL Deployment, PVC and the `Recreate` workaround. You get automated backups, patching, Multi-AZ failover and

encryption, and nothing like the volume-password mismatch you hit. Keep Flyway, since it runs from the backend and works the same way.

- **EKS:** managed Kubernetes. AWS runs the control plane, and your YAML (Deployments, Services, probes, HPA) carries over almost unchanged. The ingress rules switch from nginx to ALB annotations.
- **ECS:** AWS's own orchestrator, and an alternative to EKS rather than an addition. It uses task definitions and services instead of Kubernetes YAML.

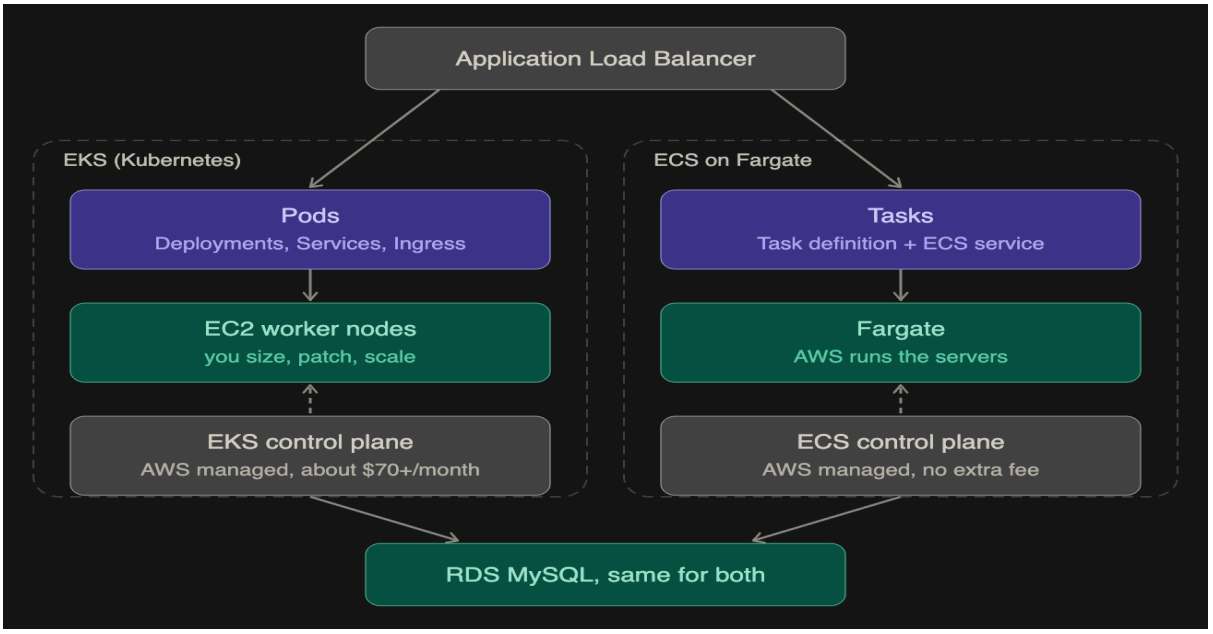
Should MySQL run on EC2 with EBS instead of RDS?

Yes it is possible, but wouldn't. You would take on backups, patching, replication and failover yourself. Use RDS unless you need engine settings it doesn't allow.

Production details worth adding

- **Security groups:** ALB opens on 443 only, node group accepts traffic from the ALB only, RDS accepts port 3306 from the node group only.
- **Frontend alternative:** host the React build on S3 with CloudFront, which is cheaper and faster than running nginx pods. The API stays behind the ALB, so the frontend calls `https://api.yourdomain.com` and you keep the CORS config.
- **Spring config:** set `SPRING_DATASOURCE_URL` to the RDS endpoint and read the credentials from Secrets Manager. Update the CORS allowed origin to your real domain.
- **Pipeline credentials:** have GitHub Actions assume an IAM role through OIDC, so no AWS keys are stored.
- **Observability:** send pod logs to CloudWatch (or Fluent Bit), and use Container Insights for metrics.

Choosing a Container Platform on AWS: EKS vs ECS on Fargate for the Vehicle Management System



1. **Workload:** the thing that runs your container. On EKS this is a pod, created by a Deployment. On ECS it is a task, created by an ECS service from a task definition.
2. **Compute:** the machine under it. On EKS you run EC2 worker nodes, so you choose the instance type, patch the OS and scale the node group. On Fargate there are no servers to see. You only say how much CPU and memory each task needs.
3. **Control plane:** the scheduler that decides what runs where. AWS manages it in both cases. EKS charges a flat hourly fee for it, and ECS does not.

The dashed arrow means “schedules and manages”. The solid arrow from the load balancer is the live traffic.

The same concepts under different names

Kubernetes (EKS)	ECS on Fargate	Project(Github Project)
Pod	Task	One Spring Boot container
Deployment	ECS service	Keeps 2 backends running, does rolling updates
`backend-deployment.yaml`	Task definition (JSON)	Image, CPU, memory, env vars, secrets
Service plus Ingress	ALB listener rules plus target group	`api.` host goes to backend, `app.` host goes to frontend
Secret	Secrets Manager reference in the task definition	`SPRING_DATASOURCE_*` values
Liveness and readiness probes	ALB health check, container health check	`/actuator/health`
`startupProbe`	`healthCheckGracePeriodSeconds`	Covers your slow Spring Boot start
HPA	ECS service auto scaling	Scale on CPU
Node	Not visible	Not needed

What you would need to change for ECS

- Convert `backend-deployment.yaml` and `frontend-deployment.yaml` into two task definitions.
- Replace the Ingress with one ALB, with a listener rule per host and one target group per service. Set the target group health check path to `/actuator/health`. This has to be allowed in Spring Security’s `permitAll()`, or the tasks will be marked unhealthy and replaced in a loop, as happened with the 403 on your probes.

- Set a health check grace period of 120 to 180 seconds, since your backend takes about a minute to start.
- Give each Spring Boot task at least 1 vCPU and 2 GB of memory. The very small sizes will slow startup the way your 500m CPU limit did.
- Keep Flyway, RDS, the Docker image and the Spring config unchanged.

With EKS, almost all of your existing YAML carries over. **Only the ingress annotations, the storage class and the secret source change.**

Cost, in plain terms

- **EKS:** a flat control-plane fee (roughly $70+ per month) plus the EC2 nodes, whether or not they are busy. Nodes can be shared by many pods, so it gets cheaper per app as you add more apps.
- **ECS on Fargate:** no control-plane fee, and you pay per task for the vCPU and memory it uses while running. For a small app with 2 or 3 tasks this is usually cheaper.
- **Shared costs in both:** the ALB, NAT Gateway, RDS and data transfer. NAT Gateways and Multi-AZ RDS are often the biggest items, so check the current AWS pricing pages before you commit.

Which to choose

If your goal is...	**Choose**
Show Kubernetes skills in a portfolio or job application	EKS
Cheapest and simplest production setup	ECS on Fargate
Many services, or a team that already knows Kubernetes	EKS
One small app, no dedicated ops person	ECS on Fargate
Possible move to another cloud later	EKS (portable YAML)

You can also run EKS pods on Fargate, which removes the nodes but keeps the Kubernetes API. It still has the control-plane fee, so it is a middle path rather than a cheap one.

EKS and ECS fargate both in AWS services?

Yes, both are AWS services, but they are not the same kind of thing.

Amazon EKS (Elastic Kubernetes Service)	AWS's managed Kubernetes	Orchestrator: decides what runs and where
Amazon ECS (Elastic Container Service)	AWS's own container orchestrator, not Kubernetes	Orchestrator: same job, different tool

AWS Fargate	Serverless compute for containers	Compute: the machines that run your containers, with no servers for you to manage

So the comparison is really two questions:

1. **Which orchestrator?** EKS (Kubernetes) or ECS (AWS-native).
2. **Which compute?** EC2 nodes (you manage them) or Fargate (AWS manages them)

All of them run in your AWS account and are billed through AWS, and they integrate with the same services: ALB, RDS, ECR, Secrets Manager, CloudWatch and IAM.